

terms of resources and is much lighter than threads. With this approach, every core can execute a million tasks per second.

Those who have run Cassandra on a large machine have probably noticed that it's not able to utilise all the available cores. Even though Cassandra is working as hard as it can, many of the cores are idle or not fully utilised. With the Scylla thread-per-core architecture, you can easily reach 100 per cent utilisation on all the cores—so you're fully utilising your hardware.

Design decision 5: Unified cache vs C* cache plethora

This was actually two design decisions wrapped into one. First, we chose not to use the Linux page cache, which is built for synchronous operation. As mentioned earlier, everything is asynchronous in Scylla, so this decision was essentially made for us. While the Linux page cache has been the recipient of a lot of great work by Linux developers, it's a general-purpose data structure. We recognised that a special-purpose cache would deliver better performance. The Linux page cache is also global, whereas Scylla is threads-per-core. We realised it would be more efficient to split the cache so that we had a separate cache for each core; thereby, we eliminated all the locking that would otherwise be required.

Due to the complexity of Cassandra's caches, it was easy deciding to build our own cache and to make it a unified one. A unified cache can dynamically tune itself to the current workload rather than having multiple different caches that need to be tuned by the user. Scylla caches objects itself, and thus always controls their eviction, their memory footprint, and can dynamically balance the different types of cache it stores. Scylla has controllers — memtable controller, compaction controls and (coming soon) cache controller — to dynamically adjust their sizes. Once the data isn't cached in memory, Scylla will generate a continuation task to read the data asynchronously from the disk using DMA. Seastar will execute it in a *usec* (1 million tasks/core/sec) and rush to run the next task. There's no blocking, no heavyweight context switch, no waste, and no tuning.

For Scylla users, this design decision means higher ratios of disk to RAM. Each node can serve a larger amount of data; so you can use smaller clusters and more fully utilise the larger disks that are available today.

Design decision 6: I/O scheduler

To manage the data on disk, both Cassandra and Scylla use an algorithm called the Log Structured Merge Tree, which relies on background operations to perform things like compaction as well as other maintenance operations, like streaming. Issues arise when these background operations compete with user queries, thereby reducing throughput—a scenario no user would opt for.

It became clear to us that we needed to control all of the I/O going through the system. The way to do this is with a scheduler that allows the types of requests—whether they are foreground requests or background requests—to be tagged.

These requests can then be metered, and the scheduler can decide the priority to be given to each class of operations.

This design decision helps Scylla users meet their SLAs. They spend less time tuning, and yet can be sure that background operations complete as quickly as possible without impacting performance. For example, if you're commissioning a new node or decommissioning an old one, you can simply tell Scylla to perform the operation without having to worry about the speed at which it happens. The operation will automatically run at the fastest speed that does not reduce system throughput.

Design decision 7: Autonomous database

This overarching design decision incorporates some (for example: Design decisions (1) and (2)) of the choices we'd already made. Our vision has always been to provide a database that dynamically tunes itself to varying conditions. We knew that Cassandra users spend a lot of their time tuning; so we decided to build a database that monitors all the operations and adjusts the rate of the internal processors to achieve a balance.

This autonomous approach made much more sense to us than saddling users with elaborate guides and explanations on how to tune for all the changing workloads that a database must contend with. Not only does an autonomous database greatly reduce the administrative burden, it also means that users can always push the database to its limits and it will always fully utilise available resources.

A foundation for success

Of course, there have been countless other decisions we've had to make. However, these seven foundational decisions set the course for everything we did thereafter. I'm happy to report that we haven't done much second guessing about our early decisions.

Now that we've reached parity with Apache Cassandra, we're soon to embark on our second wave of innovation. We have lots of ideas about how to improve Scylla, and we're happy to have many users to help guide us through this next phase.

To learn more about Scylla, read use cases and benchmarks at <http://www.scylladb.com> or try our open source database for yourself. 

References

- [1] <http://www.scylladb.com/2017/09/18/scylla-2-0-workload-conditioning/>
- [2] <http://www.scylladb.com/2017/09/21/scylla-heat-weighted-load-balancing/>

By: Dor Laor

The author is the CEO of ScyllaDB. He was part of the founding team of the KVM hypervisor under Qumranet that was acquired by Red Hat. At Red Hat, he was managing KVM and Xen development for several years. Dor holds an M.Sc from the Technion and a Ph.D in snowboarding.